Window functions

2026. February 23., Monday 10:36

Working with aggregate values, we need to use GROUP BY for all non-aggregate columns
Window functions are similar to subqueries in SELECT
These are processed after every part of the query except ORDER BY - they **use the RESULT SET!**

Window functions

- Perform an operation across a set of rows that are somehow related to the current row
- Similar to `GROUP BY` aggregate functions, but all rows remain in the output

Uses

- Fetching values from preceding or following rows (e.g. fetching the previous row's value)
  - Determining reigning champion status
  - Calculating growth over time
- Assigning ordinal ranks (1st, 2nd, *etc.*) to rows based on their values' positions in a sorted list
- Running totals, moving averages

OVER()

What's a window function?

- *How many goals were scored in each match in 2011/2012, and how did that compare to the average?*

```
SELECT
    date,
    (home_goal + away_goal) AS goals,
    AVG(home_goal + away_goal) OVER() AS overall_avg
FROM match
WHERE season = '2011/2012';
```

```
| date       | goals | overall_avg       |
|------------|-------|-------------------|
| 2011-07-29 | 3     | 2.71646           |
| 2011-07-30 | 2     | 2.71646           |
| 2011-07-30 | 4     | 2.71646           |
| 2011-07-30 | 1     | 2.71646           |
```

RANK()-column numbering highest to lowest (or lowest to highest)
A new column is needed, and OVER() clause with ORDER BY
By default, it orders from smallest to largest values - we can add DESC at the end

Generate a RANK

- *What is the rank of matches based on number of goals scored?*

```
SELECT
    date,
    (home_goal + away_goal) AS goals,
    RANK() OVER(ORDER BY home_goal + away_goal) AS goals_rank
FROM match
WHERE season = '2011/2012';
```

OVER and PARTITION BY
Splits the table into partitions based on a column's unique values - the results aren't rolled to one column
Operated on separately by the window function - ROW_NUMBER() resets for different partitions
- LAG only works if previous row is in the same p.

Partition your data

- *How many goals were scored in each match, and how did that compare to the season's average?*

```
SELECT
    date,
    (home_goal + away_goal) AS goals,
    AVG(home_goal + away_goal) OVER(PARTITION BY season) AS season_avg
FROM match;
```

We can do it for multiple columns, and it can partition aggregations, ranks,...

Anatomy of a window function

Query

```
SELECT
  Year, Event, Country,
  ROW_NUMBER() OVER () AS Row_N
FROM Summer_Medals
WHERE
  Medal = 'Gold';
```

- `FUNCTION_NAME() OVER (...)`
  - `ORDER BY`
  - `PARTITION BY`
  - `ROWS/RANGE PRECEDING/FOLLOWING/UNBOUNDED`

<u>Sliding windows-with aggregate functions</u>
Perform calculations relative to the current row
Running totals, sums, counts, averages, max-min...
It can be partitioned by one or more columns

These are frames below, always starts with ROWS/RANGE BETWEEN

Sliding window keywords

```
ROWS BETWEEN <start> AND <finish>
```

```
PRECEDING
FOLLOWING
UNBOUNDED PRECEDING
UNBOUNDED FOLLOWING
CURRENT ROW
```

1 PRECEDING

```
SELECT
    date,
    home_goal,
    away_goal,
   -- Create a running total and running average of home goals
   SUM(home_goal) OVER(ORDER BY date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
   AVG(home_goal) OVER(ORDER BY date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_avg
FROM match
```

```
FROM match
WHERE
    hometeam_id = 9908
    AND season = '2011/2012';
```

ROWS vs RANGE

- `RANGE BETWEEN [START] AND [FINISH]`
  - Functions much the same as `ROWS BETWEEN`
  - `RANGE` treats duplicates in `OVER`'s `ORDER BY` subclause as a single entity

Table

Year	Medals	Rows_RT	Range_RT
1992	10	10	10
1996	50	60	110
2000	50	110	110
2004	60	170	230
2008	60	230	230
2012	70	300	300

OVER and ORDER BY
Orders the rows related to the current row
We can order by multiple columns
We can also order outside of OVER(), which happens after the OVER(ORDER BY)

OVER and LAG
Determining reigning champion

Current and last champions

Query

```
WITH Discus_Gold AS (
  SELECT
    Year, Country AS Champion
  FROM Summer_Medals
  WHERE
    Year IN (1996, 2000, 2004, 2008, 2012)
    AND Gender = 'Men' AND Medal = 'Gold'
    AND Event = 'Discus Throw')

SELECT
  Year, Champion,
  LAG(Champion, 1) OVER
    (ORDER BY Year ASC) AS Last_Champion
FROM Discus_Gold
ORDER BY Year ASC;
```

Result

Year	Champion	Last_Champion
1996	GER	null
2000	LTU	GER
2004	LTU	LTU
2008	EST	LTU
2012	GER	EST

Pivoting

Transforming tables

Before

Country	Year	Awards
CHN	2008	74
CHN	2012	56
RUS	2008	43
RUS	2012	47
USA	2008	125
USA	2012	147

- Gold medals awarded to China, Russia, and the USA

After

Country	2008	2012
CHN	74	56
RUS	43	47
USA	125	147

- Pivoted by `Year`

CROSSTAB

Enter CROSSTAB

```
CREATE EXTENSION IF NOT EXISTS tablefunc;
```

```
SELECT * FROM CROSSTAB($$
  source_sql TEXT
$$) AS ct (column_1 DATA_TYPE_1,
           column_2 DATA_TYPE_2,
           ...,
           column_n DATA_TYPE_N);
```

"" may be needed for the column names

ROLLUP

Enter ROLLUP

```
SELECT
  Country, Medal, COUNT(*) AS Awards
FROM Summer_Medals
WHERE
  Year = 2008 AND Country IN ('CHN', 'RUS')
GROUP BY Country, ROLLUP(Medal)
ORDER BY Country ASC, Medal ASC;
```

- `ROLLUP` is a `GROUP BY` subclause that includes extra rows for group-level aggregations
- `GROUP BY Country, ROLLUP(Medal)` will count all `Country` - and `Medal` -level totals, then count only `Country` -level totals and fill in `Medal` with `null` s for these rows

ROLLUP - Query

```
SELECT
  Country, Medal, COUNT(*) AS Awards
FROM summer_medals
WHERE
  Year = 2008 AND Country IN ('CHN', 'RUS')
GROUP BY ROLLUP(Country, Medal)
ORDER BY Country ASC, Medal ASC;
```

- `ROLLUP` is hierarchical, de-aggregating from the leftmost provided column to the right-most
  - `ROLLUP(Country, Medal)` includes `Country` -level totals
  - `ROLLUP(Medal, Country)` includes `Medal` -level totals
  - Both include grand totals

ROLLUP - Result

```
| Country | Medal  | Awards |
|---------|--------|--------|
| CHN     | Bronze | 57     |
| CHN     | Gold   | 74     |
| CHN     | Silver | 53     |
| CHN     | null   | 184    |
| RUS     | Bronze | 56     |
| RUS     | Gold   | 43     |
| RUS     | Silver | 44     |
| RUS     | null   | 143    |
| null    | null   | 327    |
```

- Group-level totals contain `nulls` ; the row with all `null` s is the grand total
- Notice that it didn't include `Medal` -level totals, since it's `ROLLUP(Country, Medal)` and not `ROLLUP(Medal, Country)`

CUBE

Enter CUBE

```
SELECT
  Country, Medal, COUNT(*) AS Awards
FROM summer_medals
WHERE
  Year = 2008 AND Country IN ('CHN', 'RUS')
GROUP BY CUBE(Country, Medal)
ORDER BY Country ASC, Medal ASC;
```

- `CUBE` is a non-hierarchical `ROLLUP`
- It generates all possible group-level aggregations
  - `CUBE(Country, Medal)` counts `Country` -level, `Medal` -level, and grand totals

CUBE - Result

```
| Country | Medal  | Awards |
|---------|--------|--------|
| CHN     | Bronze | 57     |
| CHN     | Gold   | 74     |
| CHN     | Silver | 53     |
| CHN     | null   | 184    |
| RUS     | Bronze | 56     |
| RUS     | Gold   | 43     |
| RUS     | Silver | 44     |
| RUS     | null   | 143    |
| null    | Bronze | 113    |
| null    | Gold   | 117    |
| null    | Silver | 97     |
| null    | null   | 327    |
```

- Notice that `Medal` -level totals are included

- Use `ROLLUP` when you have hierarchical data (e.g., date parts) and don't want all possible group-level aggregations
- Use `CUBE` when you want all possible group-level aggregations

COALESCE

Elliminates nulls

Enter COALESCE

- `COALESCE()` takes a list of values and returns the first non-`null` value, going from left to right
- `COALESCE(null, null, 1, null, 2) → 1`
- Useful when using SQL operations that return `null` s
  - `ROLLUP` and `CUBE`
  - Pivoting
  - `LAG` and `LEAD`

Annihilating nulls

Query

```sql
SELECT
  COALESCE(Country, 'Both countries') AS Country,
  COALESCE(Medal, 'All medals') AS Medal,
  COUNT(*) AS Awards
FROM summer_medals
WHERE
  Year = 2008 AND Country IN ('CHN', 'RUS')
GROUP BY ROLLUP(Country, Medal)
ORDER BY Country ASC, Medal ASC;
```

Result

```
| Country        | Medal      | Awards |
|----------------|------------|--------|
| Both countries | All medals | 327    |
| CHN            | All medals | 184    |
| CHN            | Bronze     | 57     |
| CHN            | Gold       | 74     |
| CHN            | Silver     | 53     |
| RUS            | All medals | 143    |
| RUS            | Bronze     | 56     |
| RUS            | Gold       | 43     |
| RUS            | Silver     | 44     |
```

STRING_AGG

Data compression

Query and result

Before

```sql
WITH Country_Medals AS (
  SELECT
    Country, COUNT(*) AS Medals
  FROM Summer_Medals
  WHERE Year = 2012
    AND Country IN ('CHN', 'RUS', 'USA')
    AND Medal = 'Gold'
```

After

```sql
WITH Country_Medals AS (...),

  Country_Ranks AS (...)

  SELECT STRING_AGG(Country, ', ')
  FROM Country_Medals;
```

```sql
  AND Sport = 'Gymnastics'
GROUP BY Country),

SELECT
  Country,
  RANK() OVER (ORDER BY Medals DESC) AS Rank
FROM Country_Medals
ORDER BY Rank ASC;
```

Result

```
CHN, RUS, USA
```